2026 年竞赛作品介绍文档

店智 Copilot

面向客户的电商智能客服 Agent

售前查商品推荐 · 售后查订单退款 · 投诉升级工单

团队：邢博、刘庆昊、孔克铮

学校：河北传媒学院

专业：人工智能

提交日期：2026 年 11 月

团队信息：邢博、刘庆昊、孔克铮 / 河北传媒学院 / 人工智能

> 售前查商品推荐、售后查订单退款、投诉生成工单，一个对话框全搞定。不是只会聊天的问答工具，而是能调用工具、能校验、能溯源、敏感操作需客户确认的智能客服。

一、项目详情及使用方式

店智 Copilot 面向电商店铺的**客户**（下单前咨询、下单后查单退款投诉），解决"问商品、查订单、办售后"反复在多个页面之间切换的问题。系统把服务收进两个入口：

- **客服对话**：LangGraph 多 Agent 编排，自动判断售前/售后/投诉，调用对应工具，回答带 `[n]` 引用可溯源。
- **知识库**：43 篇运营知识文档，商家维护商品知识/售后政策，Agent 据此回答；支持切片编辑，保存后重新嵌入并失效旧检索。

项目自带 20 单模拟订单和 43 篇知识库语料，不依赖真实平台授权。启动方式：

```
pip install -r backend/requirements.txt
cp backend/.env.example backend/.env
# 填入 DEEPSEEK_API_KEY
python -m backend.app.rag.pipeline build
uvicorn backend.app.main:app --port 8000
```

另一个终端：

```
npm install
npm run dev
```

打开 `http://localhost:3000`，按“工作台 → 数据 → 对话查订单 → 退款校验 → RAG 问答 → Agent 工具调用”的顺序即可完成演示。

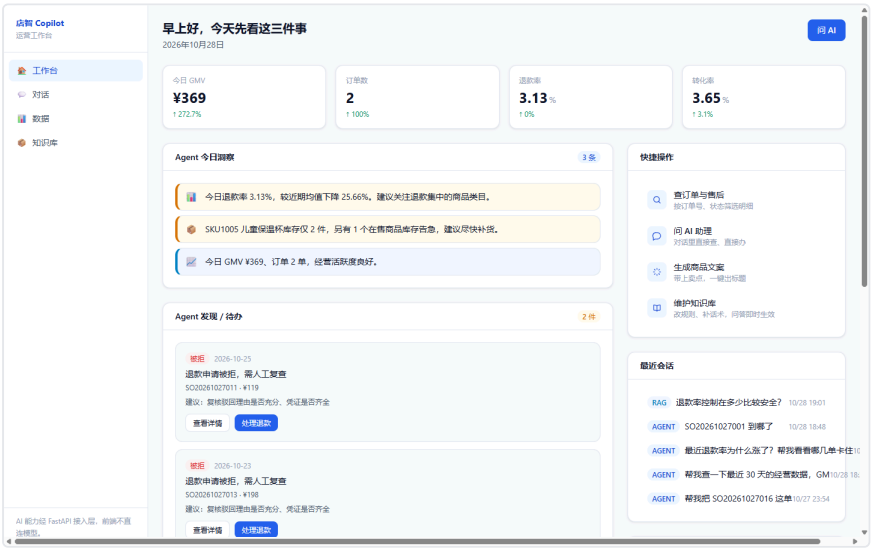


图 1：工作台首页，4 张 KPI 卡与待办



图 2：数据页订单表与状态筛选

二、项目亮点

1. **不是只会聊天的问答工具，而是能办事的工作台**
   查单、看数、退款校验、规则问答、文案生成都在同一个对话里完成。
2. **意图路由，不是所有问题都走向量检索**
   `knowledge / general` 走 RAG；`data / policy / copywriting` 走 Agent。界面用 chip 显示“数据查询 → Agent”等判断结果，用户知道系统为什么切模式。
3. **RAG 回答带 `[1][2]` 角标，来源可点开核对**
   回答不是“模型凭印象说”，而是先检索知识库，再把命中的文档片段拼进上下文。点角标能看到标题、分类和原文片段。
4. **Agent 过程透明**
   手写 ReAct 循环，最多 5 步；每次工具调用都以步骤卡展示“调了什么工具、花了多久、返回什么”。目前注册 10 个工具，覆盖知识检索、数据查询、文案、合规、订单、退款、改地址、物流拦截、退换货。
5. **敏感操作两段式门控**
   退款、改地址、物流拦截等操作统一走 `validate → 用户确认 → execute`。模型不能直接动钱；校验不通过时直接列出原因，例如“订单已超过 7 天售后期，无法自助退款”。
6. **知识库编辑即生效**
   修改切片后重新嵌入，旧检索立即失效，下一次提问就能看到新口径，不需要重启服务。
7. **真实数据可插拔**
   `DataProvider` 协议把数据源和 Agent 解耦，默认 SQLite 种子库，也可以接 MySQL、ERP API 或 CSV。
8. **安全边界清楚**
   前端永远不直连模型 API；DeepSeek Key 只放在 Python 层 `backend/.env`，所有 AI 请求走 FastAPI。



图 3：对话查订单，Agent 步骤卡展示工具调用



图 4：009 订单退款校验，红字提示超过 7 天售后期

三、AI 测试技术落地使用情况概述

项目把 AI 能力拆成可复现的测试对象，不只靠人工点页面：

测试层	内容	结果
静态检查	`tsc`、`eslint`、`next build`	全部通过
接口冒烟	7 个接口 + 4 个页面，双服务联调	**11/11 通过**
API 契约	会话、退款、知识库、意图等接口	**12/12 通过**
RAG 检索自测	10 个查询，top-3 命中	**9/10 = 90%**
RAG 效果评测 v1	20 题、40 分制、有/无 RAG 对照	**26 vs 23**
RAG 效果评测 v2	15 题、30 分制、有/无 RAG 对照	**29 vs 30**

v2 的结果没有刻意美化：通用常识题上，不带 RAG 的模型本身也能答对，所以无 RAG 反而 30/30；有 RAG 29/30。这说明 RAG 的增量主要在**平台私有规则、知识库口径、引用可溯源**，而不是所有题都靠检索。测试还暴露并修复了一个真实问题：早期 RAG 注入只截取 chunk 前 300 字，FABE 示例恰好落在 300 字之后，导致回答缺证据；改为完整原文后恢复得分。

另外还覆盖了：

- 意图分类：5 类意图 + 关键词兜底；LLM 超时/失败时降级为 `general`，不阻断对话。
- Agent 工具调用：`order_lookup`、`data_query`、`refund_flow validate/execute` 等。
- 退款门控：校验失败、重复提交、幂等执行。
- 缺 Key / 缺知识库 / 上游 503 的降级路径。
- 会话持久化、引用角标与 `/v1/rag/search` 同源核对。

测试脚本可复现：

```
node scripts/smoke.mjs
python -m backend.app.rag.selftest
python -m backend.app.rag.eval
python -m backend.app.rag.eval_v2
```



图 5：RAG 回答与来源卡，角标可回查原文

四、项目中涉及的关键技术

- **前端**：Next.js 16 App Router、TypeScript strict、Tailwind CSS、Vercel AI SDK、SSE 流式渲染。
- **后端**：Python 3.12、FastAPI、httpx、SQLite、Pydantic。
- **RAG**：jieba BM25 + fastembed BGE-small-zh 向量 + LanceDB + RRF 融合；top-4 检索；引用角标与 `/v1/rag/search` 同源。
- **Agent**：DeepSeek function calling + 手写 ReAct 循环，最多 5 步；10 个工具；SSE 事件 `step/delta/done/error`。
- **MCP**：FastMCP stdio 独立进程，复用 Agent 工具。
- **安全**：前端不直连模型；Key 只在 Python 层；同源 `/api/*` 代理；`.env` 不入库。
- **测试**：`smoke.mjs`、Pytest 风格接口脚本、RAG selftest、v1/v2 评测集、意图分类用例。



图 6：知识库切片编辑，保存后即时生效

五、项目相关的图片/照片

本文档使用 6 张真实运行截图：

1. 工作台：4 张 KPI 卡与待办（图 1）。
2. 数据页：订单表与状态筛选（图 2）。
3. 对话：查订单与 Agent 步骤卡（图 3）。
4. 退款：009 订单“超过 7 天售后期”红字拦截（图 4）。
5. RAG：回答与 `[1]` 来源卡（图 5）。
6. 知识库：切片编辑与即时生效（图 6）。

截图均来自本地演示数据，不含真实客户信息、API Key 或 `.env` 内容。

六、项目之前是否发表过或获奖

截至本次提交，本项目未公开发表、未参加其他比赛、未获奖。
如团队有其他相关获奖或发表，请在此处补充；没有则保留本段。

七、若项目是在其他项目基础上优化，请说明优化部分

本项目为原创开发，**不是基于其他参赛项目或代码库二次开发。**项目使用 Next.js、FastAPI 等开源框架和官方脚手架初始化；方案调研阶段参考了公开电商 RAG / Agent 项目的架构思路，但业务代码、演示数据、页面设计、工具实现和评测脚本均为自研。

如实际参考了某个具体开源仓库的代码，请补充仓库名、许可证和复用范围；没有代码复用则保留本段。